

The Elman RNN Formulation

The most basic RNN, introduced by Jeffrey Elman in 1990, computes the hidden state and output at each time step with two equations.

The hidden state update is:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

where:

$$\mathbf{h}_t = \tanh(\mathbf{W}[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_h)$$

- $\mathbf{h}_t \in \mathbb{R}^H$: the hidden state at time step t , a vector of dimension H (the hidden size)
- $\mathbf{h}_{t-1} \in \mathbb{R}^H$: the hidden state from the previous time step (initialized to a zero vector at $t = 0$)
- $\mathbf{x}_t \in \mathbb{R}^D$: the input at time step t , a vector of dimension D (the input size)
- $\mathbf{W}_{hh} \in \mathbb{R}^{H \times H}$: the recurrent weight matrix connecting the previous hidden state to the current one
- $\mathbf{W}_{xh} \in \mathbb{R}^{H \times D}$: the input weight matrix connecting the current input to the hidden state
- $\mathbf{b}_h \in \mathbb{R}^H$: the hidden state bias vector
- $\tanh$: the hyperbolic tangent activation function, applied element-wise to squash the values to $(-1, 1)$

$\vec{x}_1, \vec{x}_2, \dots, \vec{x}_t, \dots$ $t = 1, \dots$

$\vec{h}_t$ - скрытое состояние

$\mathbf{W} = \begin{pmatrix} \mathbf{W}_{hh} \\ \mathbf{W}_{xh} \end{pmatrix}$ $H \times D$

$H \times H$

The output at time step t (when producing a prediction at that step) is:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Linear

where:

- $\mathbf{y}_t \in \mathbb{R}^V$: the output vector at time step t (often called logits before softmax)
- $\mathbf{W}_{hy} \in \mathbb{R}^{V \times H}$: the output weight matrix
- $\mathbf{b}_y \in \mathbb{R}^V$: the output bias vector
- V : the output size (e.g., vocabulary size for language modeling)

Many-to-One (Sequence Classification)

In many-to-one mode, the RNN reads an entire sequence and produces a single output from the final hidden state. The final hidden state $\mathbf{h}_T$ serves as a compressed representation of the whole sequence, which is then passed through a linear layer to produce the prediction:

$$\hat{y} = \mathbf{W}_{hy}\mathbf{h}_T + \mathbf{b}_y$$

This configuration is appropriate for tasks where you need a single label for an entire sequence:

- **Sentiment analysis**: classify a review as positive or negative
- **Document classification**: assign a topic label to a news article
- **Spam detection**: label an email as spam or legitimate
- **Acceptability judgment**: determine whether a sentence is grammatically well-formed
- **Intent classification**: identify the intent behind a user's utterance in a dialogue system

One-to-Many (Sequence Generation)

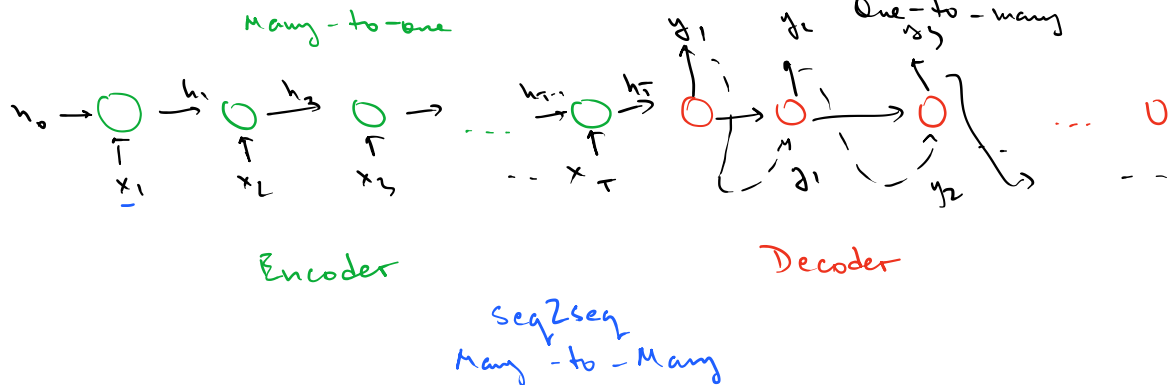
In one-to-many mode, the RNN receives a single input (or a conditioning vector) at the first step and then produces a sequence of outputs, with each output fed back as the input to the next step. This is the autoregressive generation pattern.

The typical setup uses an initial input $\mathbf{x}_0$ (perhaps a start-of-sequence token or a conditioning vector like an image embedding), then generates:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad \mathbf{x}_{t+1} = \text{embed}(\arg \max(\text{softmax}(\mathbf{y}_t)))$$

Applications of this configuration include:

- **Image captioning**: the image embedding is the initial input, and the RNN generates a caption word by word
- **Music generation**: a seed note or chord launches the generation of a melody
- **Unconditional text generation**: a start token launches free-form text
- **Code generation from a specification**: a short descriptor is encoded and the RNN expands it into a longer output

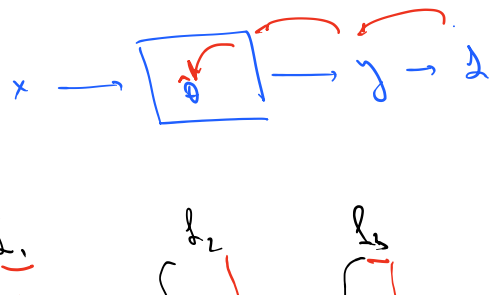


Many-to-Many (Sequence Labeling and Translation)

Many-to-many produces an output at every input step. There are two variants.

Synchronized (same-length input and output): Output $\mathbf{y}_t$ is produced from $\mathbf{h}_t$ at each step. This suits tasks where every input token needs a label, such as part-of-speech tagging or named entity recognition. The sequence labeling chapters in Part VI used CRF and HMM models for these tasks; RNNs provide an alternative that learns features directly from the text without hand-crafted feature templates.

Encoder-decoder (different-length input and output): The input sequence is first encoded into a context vector using a many-to-one RNN (the encoder),



then a one-to-many RNN (the decoder) generates the output sequence from that context. This is the foundation of sequence-to-sequence models and is explored in detail in Part IX. The classic application is machine translation: the encoder reads a sentence in English and compresses it into a fixed-size context vector, and the decoder generates the corresponding French sentence one word at a time. The attention mechanism, introduced later, addresses the bottleneck of compressing an entire sentence into a single fixed-size vector.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} \quad O(T^2)$$

$$\mathbf{w}_{hh} \quad \mathbf{w}_{xh}$$

- vanishing grad
~ 20:30
- exploding grad

The middle term $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ is the key quantity in understanding both how BPTT works and why it is numerically challenging. It is computed via the chain rule through the sequence of hidden state transitions from k to t :

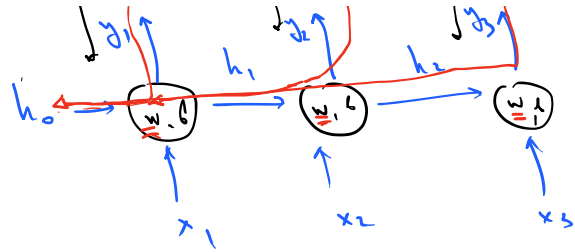
$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$$

where:

- $\prod_{j=k+1}^t$: a product of $t - k$ Jacobian matrices, one for each timestep between k and t
- $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$: the Jacobian of hidden state j with respect to hidden state $j - 1$

Each local Jacobian captures two effects: the tanh nonlinearity applied elementwise at step j , and the recurrent weight matrix $\mathbf{W}_{hh}$ that mixes hidden state dimensions. Working through the differentiation:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(1 - \mathbf{h}_j^2) \cdot \mathbf{W}_{hh}^T$$



BPTT
truncated-BPTT

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \left(\prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \cdot \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

where:

- The outer sum $\sum_{t=1}^T$ iterates over each timestep's loss
- The inner sum $\sum_{k=1}^t$ iterates over all past timesteps that contributed to $\mathbf{h}_t$ through $\mathbf{W}_{hh}$
- The product $\prod_{j=k+1}^t$ is the chain of Jacobians connecting time k to time t

The Error Signal Flowing Backward

In practice, BPTT works by defining a per-timestep error signal δ_t that captures the total gradient flowing into hidden state $\mathbf{h}_t$ from all future losses. This quantity is evaluated right to left, from $t = T$ down to $t = 1$:

$$\delta_t = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \left(\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} \right)^T \delta_{t+1}$$

where:

- $\delta_t \in \mathbb{R}^d$: the total error signal at timestep t , combining local and future gradient contributions
- $\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}$: the local gradient from the loss at time t only (obtained from the output layer's backward pass)
- $\left(\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} \right)^T \delta_{t+1}$: the gradient flowing back from the next timestep, multiplied by the transposed Jacobian
- $\delta_{T+1} = \mathbf{0}$ by convention (no loss beyond the last timestep)

① Bidirectional RNN

A Bidirectional RNN (BiRNN) runs two separate RNNs over the sequence: one processes the sequence from left to right (the forward RNN), and the other processes it from right to left (the backward RNN). The hidden states from both directions are concatenated at each position to produce the final representation.

The bidirectional hidden state at position t is:

$$\vec{\mathbf{h}}_t = \tanh(\mathbf{W}_{hh}^{(f)} \vec{\mathbf{h}}_{t-1} + \mathbf{W}_{xh}^{(f)} \mathbf{x}_t + \mathbf{b}_h^{(f)})$$

$$\overleftarrow{\mathbf{h}}_t = \tanh(\mathbf{W}_{hh}^{(b)} \overleftarrow{\mathbf{h}}_{t+1} + \mathbf{W}_{xh}^{(b)} \mathbf{x}_t + \mathbf{b}_h^{(b)})$$

$$\mathbf{h}_t^{\text{bi}} = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$$

The combined representation $\mathbf{h}_t^{\text{bi}} \in \mathbb{R}^{2H}$ encodes context from both directions. The forward and backward RNNs have independent parameters, so the total parameter count roughly doubles compared to a unidirectional RNN with the same hidden size.

Bidirectional RNNs are widely used for:

- **Named entity recognition and POS tagging**: token-level classification tasks that benefit from full sequence context
- **Question answering**: encoding a question or context passage where both left and right context matter for each token
- **Reading comprehension**: attending over a passage where understanding each span requires knowledge of the whole document
- **Pre-training language representations**: BERT and ELMo use bidirectional architectures (though BERT uses transformers rather than RNNs)

The key limitation of bidirectional RNNs is that they require the entire sequence to be available before processing begins. They cannot be used for streaming or online scenarios where you must process and respond to tokens as they arrive. For generation tasks, unidirectional RNNs are necessary.

Stacked RNNs

Just as feedforward networks gain representational power by stacking multiple layers, RNNs can be stacked by feeding the output hidden states of one RNN layer as the input to the next.

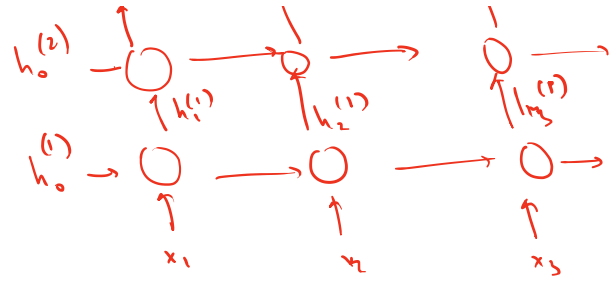
In a stacked (deep) RNN with L layers, the hidden state at layer ℓ is computed

$$\mathbf{h}_t^{(2)} = \mathbf{W}_{hh} \mathbf{h}_t^{(1)} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h$$

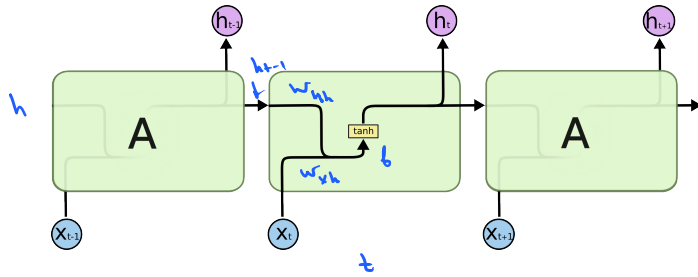
as:

$$\mathbf{h}_t^{(\ell)} = \tanh(\mathbf{W}_{hh}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{W}_{xh}^{(\ell)} \mathbf{h}_t^{(\ell-1)} + \mathbf{b}_h^{(\ell)})$$

where $\mathbf{h}_t^{(0)} = \mathbf{x}_t$ (the input) and $\mathbf{h}_t^{(\ell-1)}$ is the hidden state from the layer below at the same time step. Each layer has its own independent set of parameters.



LSTM long short-term memory 1997



Elman RNN

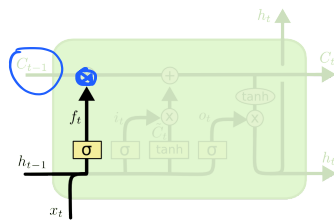
$\vec{h}_t$ hidden

$\vec{C}_t$ cell - "memory"

- forget (isobol)

- insert (godol)

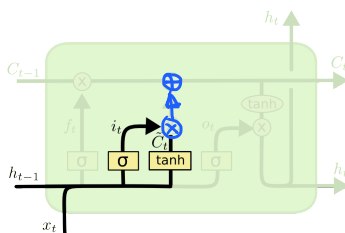
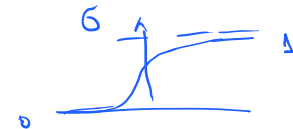
- output (godol)



forget

$$\vec{f}_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$W_{fh} \cdot h_{t-1} + W_{fx} \cdot x_t + b_f$$



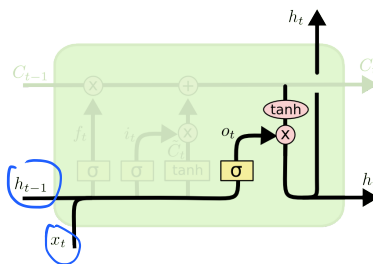
(0, 1)

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

(-1, 1)

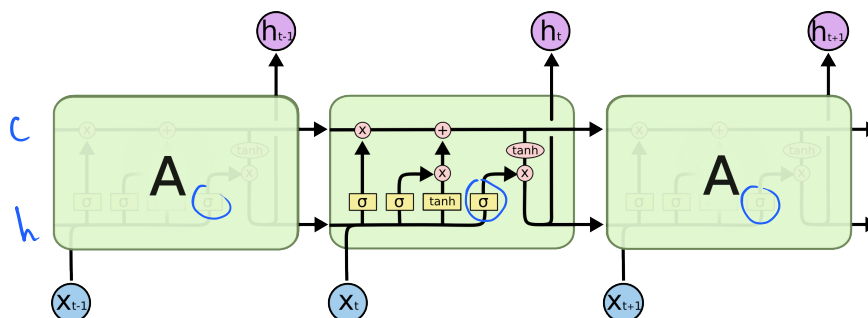
$$C_t = C_{t-1} \otimes f_t + i_t \otimes \tilde{C}_t$$



(0, 1) →

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



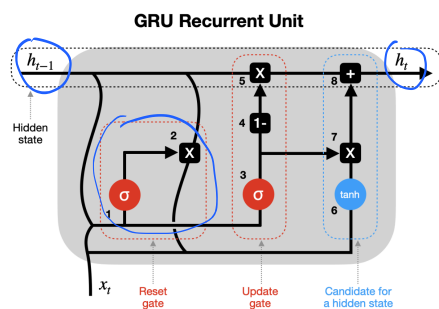
BPTT

min LSTM

min GRU

SV - State Space Model

(SSM)



h_{t-1} - hidden state at previous timestep t-1 (memory)

x_t - input vector at current timestep t

h_t - hidden state at current timestep t

$\otimes$ - vector pointwise multiplication $+$ - vector pointwise addition

$\tanh$ - tanh activation function

σ - sigmoid activation function

$\vee$ - concatenation of vectors

$\cdots$ - states

$\cdots$ - gates

$\cdots$ - updates

1-2 Reset gate - previous hidden state (h_{t-1}) and current input (x_t) are combined (multiplied by their respective weights and bias added) and passed through a reset gate. Since the sigmoid function ranges between 0 and 1, step one sets which values should be discarded (0), remembered (1), or partially retained (between 0 and 1). Step two resets the previous hidden state multiplying it with outputs from step one.

3-4-5 Update gate - step three may seem analogous to step one, but keep in mind that weights and biases used to scale these vectors are different, providing a different sigmoid output. So, after passing a combined vector through a sigmoid function, we subtract it from a vector containing all 1s (step four) and multiply it by the previous hidden state (step five). That's one part of updating the hidden state with new information.

6-7-8 Hidden state candidate - after resetting a previous hidden state in step two, the outputs are combined with new inputs (x_t), multiplying them by their respective weights and adding biases before passing through a tanh activation function (step six). Then the hidden state candidate is multiplied by the results of an update gate (step seven) and added to previously modified h_{t-1} to form the new hidden state h_t .

Next, the process repeats for timestep t+1, etc., until the recurrent unit processes the entire sequence.